

Data Science Lab (CS2311) — Exam Night Study Guide

Oct 2, 2026 · @Manav

Tonight's plan and exam strategy

All four assignments are the same pipeline with a different last step: **get data → clean it → look at it → model or test it → report**. Learn one reusable template per stage and you can adapt it to any question they set tomorrow.

The pipeline in one line: HTML → (BeautifulSoup) → list of dicts → JSON → DataFrame → clean → features → scale → plots → model / test → metrics table → CSV + 5 insights.

Time-boxed plan (about 7 hours)

Block	Time	What to do	Output you should be able to write from memory
1	0:00–1:00	Module 1 (scraping). Run the campus-events example below on the given HTML file, then the books.toscraper loop	<code>soup.find_all</code> , <code>.find(...).text</code> , handling a missing tag, <code>json.dump</code> , <code>pd.DataFrame</code>
2	1:00–2:15	Module 2 (wrangling + plots) on the scraped books data	<code>isnull().sum()</code> , <code>fillna</code> , <code>drop_duplicates</code> , <code>str.replace</code> + <code>astype</code> , 3 new features, <code>MinMaxScaler</code> , 5 plots, <code>to_csv</code>
3	2:15–3:15	Module 3 (California housing, MLR, RFE)	<code>fetch_california_housing</code> , IQR outliers, <code>train_test_split</code> , <code>LinearRegression</code> , MAE/MSE/RMSE/R ² , RFE
4	3:15–4:15	Module 4 (classification, churn)	<code>get_dummies</code> , 5 classifiers in a loop, confusion matrix, ROC + PR curves, metrics table
—	4:15–4:30	Break	

Block	Time	What to do	Output you should be able to write from memory
5	4:30–5:15	Module 5 (PCA, SVD)	<code>PCA(n_components=0.95)</code> , <code>TruncatedSVD</code> , explained variance plot, re-run the classifier loop
6	5:15–6:00	Module 6 (Ridge, Lasso, ElasticNet)	alpha loop, train/test error vs alpha plot, coefficient comparison
7	6:00–6:30	Module 7 (t-tests)	<code>ttest_ind</code> , <code>ttest_rel</code> , the decision rule
8	6:30–7:00	Cheat sheet + viva questions; type one template from memory without looking	

Exam strategy

- **Reuse, don't reinvent.** Every module below ends with a template. In the exam, copy the skeleton and change only the column names and dataset.
- **Print as you go.** After every step print `df.shape` , `df.head()` , `df.isnull().sum()` . Examiners award marks for showing each pipeline stage.
- **Write markdown cells.** One line before each code cell saying what and why ("Min-Max scaling because price has a bounded range and no strong outliers"). Justifications carry marks.
- **Always finish with a comparison table + a conclusion sentence.** Nearly every question asks "which is best and why".
- **If a website blocks you (IMDb often does), say so and fall back.** Use a saved HTML file or a Kaggle CSV of the same data, and state it in a markdown cell. Never lose the whole question to a 403 error.
- **Set `random_state=42` everywhere** so your numbers are reproducible.

What carries the most marks (from the marking scheme): Assignment 1 Q3 is out of 75, so the recipe-style "build a dataset from scratch and answer 6–8 analytical questions" type deserves practice. The analytical questions are almost all `groupby` + `value_counts` + a bar chart.

Module 1: Web scraping → JSON → DataFrame

Scraping is just "find the repeating box, then pull the same fields out of every box". Once you see that pattern, every site is the same code.

Intuition first

Analogy: A web page is like a stack of identical visiting cards (each `<div class="event">`). Each card has the same slots (name, venue, fee). You loop over the cards and copy each slot into a dictionary. A list of those dictionaries IS your JSON, and `pd.DataFrame(list_of_dicts)` turns it into a table.

Three data shapes the assignment wants you to show:

- **Unstructured / HTML:** raw text with tags. Machine-readable only after parsing.
- **Semi-structured / JSON:** key-value pairs, can be nested, fields may be missing per record.
- **Structured / DataFrame:** fixed columns, one row per record, one type per column.

The 6 BeautifulSoup calls you need

Call	What it does
<code>BeautifulSoup(html, 'html.parser')</code>	Parse HTML text into a searchable tree
<code>soup.find('h2')</code>	First matching tag (or <code>None</code> if absent)
<code>soup.find_all('div', class_='event')</code>	List of all matching tags (note the underscore in <code>class_</code>)
<code>tag.get_text(strip=True)</code>	The text inside, whitespace trimmed
<code>tag['href']</code> or <code>tag.get('href')</code>	An attribute value (links, image src, class)
<code>soup.select('article.product_pod h3 a')</code>	CSS selector version of <code>find_all</code>

Golden rule: `find()` returns `None` when a tag is missing, and `None.get_text()` crashes. Always wrap it in a helper.

Worked example: your Campus Events HTML

The last event (Career Guidance Talk) has **no venue tag**. That is deliberate: it tests whether your scraper survives a missing field and whether you handle the missing value later.

```
from bs4 import BeautifulSoup
import json
import pandas as pd

# 1. Load HTML (from a file here; from requests.get(url).text for a website)
html = open('HTML_Reference_Sample_Campus_Events.html', encoding='utf-8').read()
soup = BeautifulSoup(html, 'html.parser')

# 2. Safe helper: returns None instead of crashing when a tag is missing
def get_text(parent, tag, cls):
    el = parent.find(tag, class_=cls)
    return el.get_text(strip=True) if el else None

# 3. Loop over the repeating box
events = []
for div in soup.find_all('div', class_='event'):
    events.append({
        'name':    div.find('h2').get_text(strip=True),
        'venue':   get_text(div, 'p', 'venue'),
        'date':    get_text(div, 'p', 'date'),
        'category': get_text(div, 'span', 'category'),
        'fee':     get_text(div, 'span', 'fee'),
        'seats':   get_text(div, 'p', 'seats'),
    })

# 4. HTML -> JSON (semi-structured)
with open('events.json', 'w', encoding='utf-8') as f:
    json.dump(events, f, indent=2, ensure_ascii=False)    # ensure_ascii=False
                                                         keeps the rupee sign

# 5. JSON -> DataFrame (structured)
with open('events.json', encoding='utf-8') as f:
    df = pd.DataFrame(json.load(f))
print(df.isnull().sum())    # venue: 1  <- the Career talk

# 6. Clean
```

```

df['venue'] = df['venue'].fillna('Unknown')
df['fee']    = df['fee'].str.replace('₹', '', regex=False).replace('Free',
'0').astype(int)
df['seats'] = df['seats'].str.extract(r'(\d+)').astype(int)      # 'Seats:
30' -> 30
df['date']  = pd.to_datetime(df['date'], format='%d %B %Y')      # '12 October
2026'

# 7. New features
df['is_free']      = df['fee'].eq(0)
df['day_of_week']  = df['date'].dt.day_name()
df['revenue_potential'] = df['fee'] * df['seats']

```

Dry run of the tricky lines:

- '₹250' → `str.replace('₹', '')` → '250' → `astype(int)` → 250.
- 'Free' → replace does nothing → `.replace('Free', '0')` (whole-value match) → '0' → 0.
- 'Seats: 45' → regex `(\d+)` captures the digits → '45' → 45.
- Result: 7 rows, 1 missing venue filled; Python Workshop has the highest revenue potential ($250 \times 45 = ₹11,250$).

books.toscrape.com template (Assignment 1 Q1)

The site has 50 listing pages × 20 books = 1000 books. Listing pages give title, price, rating, availability and link; each book's own page gives UPC, category, description and stock count.

```

import requests, time, json
from bs4 import BeautifulSoup
from urllib.parse import urljoin

BASE = 'https://books.toscrape.com/catalogue/'
books = []

for page in range(1, 51):                                # 50 pages
    url = f'{BASE}page-{page}.html'
    soup = BeautifulSoup(requests.get(url).content, 'html.parser')

    for art in soup.find_all('article', class_='product_pod'):
        link = urljoin(url, art.h3.a['href'])              # relative -> absolute URL
        book = {

```

```

        'title':      art.h3.a['title'],    # full title is in the
attribute, the text is truncated
        'price':      art.find('p', class_='price_color').text,
        'rating':      art.find('p', class_='star-rating')['class'][1],
# ['star-rating', 'Three'] -> 'Three'
        'availability': art.find('p', class_='instock
availability').get_text(strip=True),
        'url':        link,
    }

    # detail page for the remaining attributes
    d = BeautifulSoup(requests.get(link).content, 'html.parser')
    book['category'] = d.select('ul.breadcrumb li a')[2].text
    desc = d.find('div', id='product_description')
    book['description'] = desc.find_next_sibling('p').text if desc else
None
    table = {row.th.text: row.td.text for row in d.select('table.table
tr')}}
    book['upc']      = table.get('UPC')
    book['stock']    = table.get('Availability')          # 'In stock (22
available)'
    book['reviews']  = table.get('Number of reviews')
    books.append(book)
    time.sleep(0.5)                                     # be polite to the server

    json.dump(books, open('books.json', 'w'), indent=2)

```

Why `.content` instead of `.text` : the site's price contains £, and `.content` (bytes) lets BeautifulSoup detect the encoding correctly. With `.text` you may see £51.77 ; if you do, strip both characters.

Exam shortcut: 1000 detail-page requests take a few minutes. If time is short, scrape only listing pages first (fast), show the DataFrame, then add the detail loop with a `[:N]` slice.

IMDb Top 250 (Assignment 1 Q2) — what to expect

- IMDb blocks plain requests. Always send a browser header: `requests.get(url, headers={'User-Agent': 'Mozilla/5.0', 'Accept-Language': 'en-US'})`.
- The modern page stores all 250 movies as JSON inside `<script type="application/ld+json">`. Easiest path: `json.loads(soup.find('script', type='application/ld+json').string)['itemListElement']`. That gives name, rating, votes, duration, genre, certificate, URL directly.

- If you get a 403 or fewer than 250 rows, state it in a markdown cell and load a Kaggle IMDb Top 250 CSV instead. The wrangling marks are still fully available.

Recipe dataset (Assignment 1 Q3) — strategy

- No single site has all 200 recipes with every field. Scrape what you can (name, ingredients, time, image, URL) from 1-2 recipe sites, and add region, state, veg/non-veg and GI tag through a **lookup dictionary** you build (e.g. `state_to_region = {'Punjab': 'North', 'Kerala': 'South', ...}`).
- Say clearly in the report which fields were scraped and which were derived. That honesty is worth marks.
- Duplicates across sources: normalise the name first (`str.lower().str.strip()`), then `drop_duplicates(subset='name_clean')`.

Module 2: Data wrangling, features, normalization, plots

Wrangling is a fixed checklist: **missing** → **duplicates** → **text** → **types** → **new features** → **scaling** → **plots** → **save**. Do it in that order every time and say which step each cell is.

The checklist with the one-liner for each

Step	Code	Why
Inspect	<code>df.shape, df.info(), df.describe(), df.head()</code>	Know sizes and types before touching anything
Find missing	<code>df.isnull().sum()</code>	Count per column
Fill numeric	<code>df['col'] = df['col'].fillna(df['col'].median())</code>	Median is robust to outliers (mean is pulled by extremes)
Fill categorical	<code>df['col'] = df['col'].fillna(df['col'].mode()[0])</code> or <code>'Unknown'</code>	Mode keeps the most common category
Drop rows	<code>df.dropna(subset=['title'])</code>	Only when the key field is missing

Step	Code	Why
Duplicates	<code>df.duplicated().sum()</code> , then <code>df.drop_duplicates(subset='upc')</code>	Use a unique ID column if one exists
Text cleanup	<code>df['col'].str.strip().str.lower() /</code> <code>.str.title()</code>	'punjab', 'Punjab' → one category
Price to number	<code>df['price'].str.replace('£', '').str.replace('Â', '').astype(float)</code>	Strings can't be averaged
Words to number	<code>df['rating'].map({'One':1, 'Two':2, 'Three':3, 'Four':4, 'Five':5})</code>	Ordinal mapping
Extract number	<code>df['stock'].str.extract(r'(\d+)').astype(int)</code>	'In stock (22 available)' → 22
Votes like '2.9M'	write a small function (below)	Suffix to multiplier
Category type	<code>df['category'].astype('category')</code>	Saves memory, signals intent
Save	<code>df.to_csv('books_clean.csv', index=False)</code>	<code>index=False</code> avoids an extra unnamed column

```
def parse_votes(v):          # '2.9M' -> 2900000, '850K' -> 850000
    v = str(v).strip().upper()
    if v.endswith('M'): return float(v[:-1]) * 1_000_000
    if v.endswith('K'): return float(v[:-1]) * 1_000
    return float(v.replace(',',''))
df['votes'] = df['votes'].apply(parse_votes)

def parse_runtime(s):        # '2h 22m' -> 142 minutes
    h = m = 0
    for part in str(s).split():
        if part.endswith('h'): h = int(part[:-1])
        if part.endswith('m'): m = int(part[:-1])
    return h * 60 + m
```


Feature engineering — ready-made features per assignment

Feature engineering: creating a new column from existing ones that captures something a model or reader cares about.

Dataset	Feature	Code
Books	price_category	<code>pd.cut(df.price, bins=[0,20,40,60], labels=['Low', 'Medium', 'High'])</code>
Books	title_length	<code>df.title.str.len()</code>
Books	in_stock (0/1)	<code>df.availability.str.contains('In stock').astype(int)</code>
Books	price_per_rating	<code>df.price / df.rating</code>
Movies	movie_age	<code>2026 - df.year</code>
Movies	decade	<code>(df.year // 10) * 10</code>
Movies	rating_category	<code>pd.cut(df.rating, [0,8.0,8.5,10], labels=['Good', 'Great', 'Masterpiece'])</code>
Movies	runtime_category	<code>pd.cut(df.runtime, [0,100,150,400], labels=['Short', 'Medium', 'Long'])</code>
Recipes	ingredient_count	<code>df.ingredients.str.split(',').str.len()</code>
Recipes	spice_level	count of words like chilli, pepper, garam masala in ingredients
Recipes	complexity	<code>pd.cut(df.prep_time, [0,20,45,500], labels=['Easy', 'Medium', 'Hard'])</code>

Dry run of `pd.cut` : bins `[0,20,40,60]` make intervals `(0,20]`, `(20,40]`, `(40,60]`. Price 51.77 falls in `(40,60]` → 'High'. Price exactly 20 → 'Low' (right edge included).

Normalization — which one and how to justify it

Intuition: Scaling puts columns on the same ruler. Without it, a column measured in thousands (votes) drowns out one measured in single digits (rating) in any distance-based or gradient-based method.

Technique	Formula (plain)	Range after	Use when	Avoid when
Min-Max	$x' = (x - \min) / (\max - \min)$	[0, 1]	Bounded data, no big outliers (price £10–£60, rating 1–5), neural nets, KNN	Strong outliers squash everything else near 0
Standard (Z-score)	$z = (x - \text{mean}) / \text{std}$	mean 0, std 1	Roughly bell-shaped data; linear/logistic regression, PCA, SVM	— (safe default)
Robust	$(x - \text{median}) / \text{IQR}$	centred at 0	Heavy outliers (votes, gross, population)	—
Log	$\log(1 + x)$	compressed	Right-skewed counts (votes, gross revenue) — often before another scaler	Data with negatives

Sample justification sentence (Books): "Price lies in a bounded range (£10–£60) with no extreme outliers, so Min-Max scaling maps it to [0,1] while preserving the shape of the distribution and making it comparable with the rating scale."

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler
df['price_norm'] = MinMaxScaler().fit_transform(df[['price']])      # double
brackets = 2-D input
```

Dry run of Min-Max: min = 10, max = 60, x = 35 → $(35 - 10) / (60 - 10) = 25 / 50 = 0.5$.

Visualizations — a plot for every question type

Question asked	Plot	Code (seaborn as sns, matplotlib.pyplot as plt)
Distribution of one number	Histogram	<code>sns.histplot(df.price, bins=20, kde=True)</code>
Count per category	Count / bar plot	<code>sns.countplot(x='rating', data=df)</code>
Top-N categories	Horizontal bar	<code>df.category.value_counts().head(10).plot(kind='barh')</code>

Question asked	Plot	Code (seaborn as sns, matplotlib.pyplot as plt)
Number across categories	Box plot	<code>sns.boxplot(x='rating', y='price', data=df)</code>
Two numbers	Scatter	<code>sns.scatterplot(x='votes', y='rating', data=df)</code>
All numeric relations	Heatmap	<code>sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')</code>
Proportions	Pie	<code>df.veg.value_counts().plot(kind='pie', autopct='%1.1f%%')</code>
Trend over time	Line	<code>df.groupby('decade').size().plot(marker='o')</code>
Category × category	Grouped bar / heatmap	<code>pd.crosstab(df.region, df.meal_type).plot(kind='bar')</code>

Always add `plt.title(...)`, axis labels and `plt.show()`, and write one insight sentence under each plot.

Answering "analytical questions" (Recipe Q3 style)

They are all groupby / counting:

- Region with most variety → `df.groupby('region')['name'].nunique().sort_values(ascending=False)`
- Most common ingredients → `df.ingredients.str.split(',').explode().str.strip().str.lower().value_counts().head(15)`
- Most common cooking method → `df.cooking_method.value_counts()`
- States with most GI tags → `df[df.gi_tag == 'Yes'].state.value_counts()`
- Veg vs non-veg share → `df.veg_type.value_counts(normalize=True) * 100`
- Dominant meal per region → `pd.crosstab(df.region, df.category).idxmax(axis=1)`

explode() dry run: a row with `['rice', 'dal', 'ghee']` becomes three rows, one ingredient each, so `value_counts` counts ingredients across all recipes.

Writing the 5 business insights

Formula: **observation + number + so-what**. Example: "Five-star books are not more expensive (mean £A vs £B for one-star — put your own numbers here), so pricing is not quality-driven; the store could test a premium for highly rated titles." Compute the numbers with `groupby().mean()` so every insight is backed by output.

Module 3: EDA + Multiple Linear Regression (California Housing)

The whole assignment is: explore → clean → split → fit three models on different feature sets → compare with 4 metrics + time → pick the best, then repeat with feature selection.

Intuition first

Linear regression analogy: You are pricing a house with a formula like $\text{price} = 0.4 \times \text{income} + 0.01 \times \text{house_age} - 0.1 \times \text{latitude} + \dots$. The model learns those weights (coefficients) so the predictions are as close as possible to the real prices, by minimising the sum of squared errors.

Multiple Linear Regression: $y = b_0 + b_1 \cdot x_1 + b_2 \cdot x_2 + \dots + b_n \cdot x_n$. "Multiple" just means more than one input feature.

Dataset facts (memorise): 20,640 rows, 8 numeric features, no missing values. Target `MedHouseVal` is in units of \$100,000 and is capped at 5.0. `MedInc` (median income) has by far the strongest correlation with the target (about 0.69). `AveRooms`, `AveBedrms`, `AveOccup`, `Population` have big outliers.

Regression metrics

Metric	Formula (plain)	Meaning	Better
MAE	mean of $\text{abs}(y - \hat{y})$	Average error in target units	Lower
MSE	mean of $(y - \hat{y})^2$	Punishes big errors heavily	Lower
RMSE	$\text{sqrt}(\text{MSE})$	Like MAE but punishes big errors; back in target units	Lower
R^2	$1 - \text{SS}_{\text{res}} / \text{SS}_{\text{tot}}$	Fraction of variance explained (1 = perfect, 0 = as good as predicting the mean)	Higher

Dry run: $y = [3, 5]$, $\hat{y} = [2, 7]$. Errors = $[1, -2]$. MAE = $(1+2)/2 = 1.5$. MSE = $(1+4)/2 = 2.5$. RMSE = 1.58.

EDA + preprocessing template

```
import pandas as pd, numpy as np, matplotlib.pyplot as plt, seaborn as sns,
time
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

data = fetch_california_housing(as_frame=True)
df = data.frame                                # 8 features + 'MedHouseVal'
# (offline? df = pd.read_csv('housing.csv') from Kaggle - column names
differ)

# --- EDA ---
print(df.shape); print(df.dtypes); print(df.describe())
print(df.isnull().sum()); print(df.duplicated().sum())

df.hist(bins=40, figsize=(14, 10)); plt.show()
sns.heatmap(df.corr(), annot=True, fmt='.2f', cmap='coolwarm'); plt.show()
print(df.corr()['MedHouseVal'].sort_values(ascending=False))
sns.scatterplot(x='MedInc', y='MedHouseVal', data=df, alpha=0.2); plt.show()
plt.scatter(df.Longitude, df.Latitude, c=df.MedHouseVal, cmap='viridis',
s=2); plt.colorbar(); plt.show()
for c in ['AveRooms', 'AveBedrms', 'Population', 'AveOccup']:
    sns.boxplot(x=df[c]); plt.title(c); plt.show()

# --- Outliers: IQR capping (keeps all rows) ---
for c in ['AveRooms', 'AveBedrms', 'Population', 'AveOccup']:
    Q1, Q3 = df[c].quantile([0.25, 0.75])
    IQR = Q3 - Q1
    df[c] = df[c].clip(Q1 - 1.5 * IQR, Q3 + 1.5 * IQR)

# --- Split, then scale (fit scaler on TRAIN only) ---
X = df.drop('MedHouseVal', axis=1); y = df['MedHouseVal']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
```

```

random_state=42)
scaler = StandardScaler()
X_train_s = pd.DataFrame(scaler.fit_transform(X_train), columns=X.columns)
X_test_s = pd.DataFrame(scaler.transform(X_test), columns=X.columns)

```

IQR dry run: $Q1 = 4$, $Q3 = 6 \rightarrow IQR = 2 \rightarrow$ fences $4 - 3 = 1$ and $6 + 3 = 9$. A value of 140 is clipped to 9; a value of 5 stays.

Why fit the scaler only on train: using test-set min/mean leaks future information into training (data leakage), making scores look better than they really are.

Three models (Assignment 2 Q2)

```

corr = df.corr()
['MedHouseVal'].drop('MedHouseVal').abs().sort_values(ascending=False)
sets = {
    'A (top 3 corr)': corr.index[:3].tolist(),
    'B (5 chosen)': ['MedInc', 'AveRooms', 'HouseAge', 'Latitude',
'Longitude'], # justify: income + size + age + location
    'C (all 8)': X.columns.tolist(),
}

rows = []
for name, feats in sets.items():
    m = LinearRegression()
    t0 = time.perf_counter(); m.fit(X_train_s[feats], y_train); t =
time.perf_counter() - t0
    p = m.predict(X_test_s[feats])
    mse = mean_squared_error(y_test, p)
    rows.append([name, len(feats), mean_absolute_error(y_test, p), mse,
np.sqrt(mse), r2_score(y_test, p), t])

results = pd.DataFrame(rows, columns=['Model', '#Features', 'MAE', 'MSE',
'RMSE', 'R2', 'Train time (s)'])
print(results)

```

What to expect and write: Model C usually gives the best R^2 (around 0.6). Latitude and Longitude only help together, which is why picking them in Model B improves it a lot over Model A. Training time grows with features but stays in milliseconds — **conclusion:** for linear regression the computational cost is negligible, so the accuracy gain of more informative features is worth it; on huge datasets, fewer features reduce cost and overfitting risk.

Feature selection (Assignment 2 Q3)

Three methods:

- **Correlation filter:** keep features with |corr with target| above a threshold (e.g. 0.1). Fast, but ignores interactions.
- **Coefficient magnitude:** on SCALED data, a bigger |coefficient| = more influence.
`pd.Series(m.coef_, index=feats).abs().sort_values()`
- **RFE (Recursive Feature Elimination):** fit the model, drop the weakest feature, refit, repeat until k remain.

```
from sklearn.feature_selection import RFE
rfe = RFE(LinearRegression(), n_features_to_select=5).fit(X_train_s, y_train)
selected = X.columns[rfe.support_].tolist()
print('Selected:', selected)
print(pd.Series(rfe.ranking_, index=X.columns).sort_values()) # 1 = kept
# then refit LinearRegression on selected and add a row to the comparison table
```

RFE dry run (k = 2, 4 features): fit on [a, b, c, d] → |coef| smallest is c → drop c. Fit on [a, b, d] → smallest is a → drop a. Left with [b, d].

Conclusion template: "Reducing from 8 to 5 features (RFE) changed R^2 from X to Y. Since the drop is small (or zero), the 5-feature model is preferred: it is simpler, faster and less prone to overfitting with nearly the same accuracy." If R^2 drops noticeably, say all features are better because each carries unique information.

Module 4: Classification (Telco Customer Churn)

Classification predicts a label (Churn: Yes/No) instead of a number. The work is: clean → encode → scale → split → loop over 5 models → confusion matrix + 5 metrics + ROC + PR curves → comparison table.

Intuition: the confusion matrix

Analogy: A doctor screening for a disease. "Positive" = churn = the thing you're hunting for.

	Predicted No	Predicted Yes
Actually No	TN (correctly ignored)	FP (false alarm)

	Predicted No	Predicted Yes
Actually Yes	FN (missed churner — costly!)	TP (caught churner)

sklearn prints it in exactly this layout: `[[TN, FP], [FN, TP]]`.

Metric	Formula	Question it answers
Accuracy	$(TP + TN) / \text{total}$	How often am I right overall? Misleading when classes are imbalanced
Precision	$TP / (TP + FP)$	When I say "churn", how often am I right?
Recall	$TP / (TP + FN)$	Of all real churners, how many did I catch?
F1	$2 \cdot P \cdot R / (P + R)$	Balance of precision and recall
ROC-AUC	area under TPR vs FPR curve	Probability the model ranks a random churner above a random non-churner (0.5 = coin flip, 1 = perfect)

Dry run: TP = 60, FN = 40, FP = 20, TN = 280 (400 customers). Accuracy = $340/400 = 0.85$. Precision = $60/80 = 0.75$. Recall = $60/100 = 0.60$. F1 = $2 \cdot 0.75 \cdot 0.60 / 1.35 = 0.667$.

Key viva point: about 26% of Telco customers churn. A model that always says "No" gets about 74% accuracy and catches zero churners. That is why recall, F1 and ROC-AUC matter more than accuracy here.

ROC vs PR curve: ROC plots recall (TPR) against false-positive rate over all thresholds. The PR curve plots precision against recall and is more honest on imbalanced data because it ignores the many easy true negatives.

Dataset quirks (memorise these — they are the "traps")

- `customerID` is a unique ID → drop it.
- `TotalCharges` is stored as text and has 11 blank strings (new customers with tenure 0) → `pd.to_numeric(..., errors='coerce')` turns blanks into NaN, then fill with 0 or the median.
- `Churn` is 'Yes'/'No' → map to 1/0.
- `SeniorCitizen` is already 0/1.
- Many columns have three values like 'Yes', 'No', 'No internet service'.

Full template

```
import pandas as pd, numpy as np, matplotlib.pyplot as plt, seaborn as sns,
time
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.metrics import (confusion_matrix, ConfusionMatrixDisplay,
accuracy_score, precision_score,
                             recall_score, f1_score, roc_auc_score,
roc_curve, precision_recall_curve,
                             average_precision_score)

df = pd.read_csv('WA_Fn-UseC_-Telco-Customer-Churn.csv')

# --- Clean ---
df = df.drop('customerID', axis=1).drop_duplicates()
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
df['TotalCharges'] = df['TotalCharges'].fillna(df['TotalCharges'].median())
df['Churn'] = df['Churn'].map({'Yes': 1, 'No': 0})

# --- EDA ---
sns.countplot(x='Churn', data=df); plt.title('Target distribution');
plt.show()
sns.countplot(x='Contract', hue='Churn', data=df); plt.show()           #
month-to-month churns most
sns.boxplot(x='Churn', y='tenure', data=df); plt.show()                 #
churners have low tenure
sns.histplot(data=df, x='MonthlyCharges', hue='Churn', kde=True); plt.show()

# --- Encode categoricals (one-hot) ---
df = pd.get_dummies(df, drop_first=True, dtype=int)

# --- Split (stratify keeps the 26% churn ratio in both sets) + scale ---
X = df.drop('Churn', axis=1); y = df['Churn']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42, stratify=y)
sc = StandardScaler()
```

```

X_train = sc.fit_transform(X_train); X_test = sc.transform(X_test)

models = {
    'Logistic Regression': LogisticRegression(max_iter=1000),
    'KNN':                  KNeighborsClassifier(n_neighbors=7),
    'Decision Tree':        DecisionTreeClassifier(max_depth=5,
random_state=42),
    'Random Forest':        RandomForestClassifier(n_estimators=200,
random_state=42),
    'Naive Bayes':          GaussianNB(),
    'SVM':                  SVC(probability=True, random_state=42),    #
probability=True needed for ROC
}

def evaluate(models, X_train, X_test, y_train, y_test, tag=''):
    rows = []
    fig, ax = plt.subplots(1, 2, figsize=(14, 5))
    for name, m in models.items():
        t0 = time.perf_counter(); m.fit(X_train, y_train); t =
time.perf_counter() - t0
        pred = m.predict(X_test)
        proba = m.predict_proba(X_test)[: , 1]          # probability of class
1 (churn)

        ConfusionMatrixDisplay(confusion_matrix(y_test, pred)).plot();
plt.title(f'{name} {tag}'); plt.show()

        rows.append([name, accuracy_score(y_test, pred),
precision_score(y_test, pred),
                    recall_score(y_test, pred), f1_score(y_test, pred),
roc_auc_score(y_test, proba), t])

        fpr, tpr, _ = roc_curve(y_test, proba)
        ax[0].plot(fpr, tpr, label=f'{name} (AUC={roc_auc_score(y_test,
proba):.2f})')
        prec, rec, _ = precision_recall_curve(y_test, proba)
        ax[1].plot(rec, prec, label=f'{name} (AP=
{average_precision_score(y_test, proba):.2f})')

        ax[0].plot([0, 1], [0, 1], 'k--'); ax[0].set(xlabel='FPR', ylabel='TPR',
title=f'ROC {tag}'); ax[0].legend()
        ax[1].set(xlabel='Recall', ylabel='Precision', title=f'Precision-Recall
{tag}'); ax[1].legend()

```

```
plt.show()
res = pd.DataFrame(rows, columns=['Model', 'Accuracy', 'Precision',
'Recall', 'F1', 'ROC-AUC', 'Train time (s)'])
return res.sort_values('ROC-AUC', ascending=False)

results_orig = evaluate(models, X_train, X_test, y_train, y_test,
'(original)')
print(results_orig)
results_orig.set_index('Model')[['Accuracy', 'Precision', 'Recall', 'F1',
'ROC-AUC']].plot(kind='bar', figsize=(12, 5)); plt.show()
```

Why write `evaluate()` as a function: Assignment 3 Q2 asks you to repeat the exact same experiment on PCA and SVD features. With the function, that is two extra lines.

The 5 classifiers in one line each

- **Logistic Regression:** a linear score squashed through a sigmoid into a probability 0–1; threshold 0.5. Usually the best or near-best on Telco.
- **KNN:** look at the k nearest customers and take a vote. Needs scaling because it uses distance.
- **Decision Tree:** a flowchart of yes/no questions ("contract = month-to-month?"). Easy to explain, overfits without `max_depth`.
- **Random Forest:** many trees on random subsets, majority vote. Less overfitting than one tree.
- **Naive Bayes:** applies Bayes' rule assuming features are independent. Very fast, often high recall, lower precision.
- **SVM:** finds the widest-margin boundary between classes. Slowest to train here.

Conclusion template: "Logistic Regression achieved the highest ROC-AUC (X) and F1 (Y). Since churn is the minority class and missing a churner is costly, we rank by ROC-AUC and recall rather than accuracy."

Module 5: PCA and SVD (dimensionality reduction)

PCA and SVD both rotate the data onto new axes ordered by how much variance they capture, then keep only the first few axes. Fewer columns, most of the information.

Intuition first

Analogy: Photographing a 3-D object for an ID card. You turn it until the photo shows the most detail (the widest spread). That best viewing direction is principal component 1. The next-best direction at right angles to it is PC2. Throw away directions where the object is almost flat — you lose little.

Why it helps: Telco has around 30 columns after one-hot encoding, and many are correlated (e.g. 'No internet service' appears in six columns). PCA merges correlated columns into a few uncorrelated components → faster training, less noise. The cost: components are mixtures of original columns, so they are hard to interpret.

Explained variance ratio: the fraction of total variance each component captures. Choose k so the cumulative ratio reaches about 0.90–0.95.

The algorithms (what "implement" means)

PCA (eigen-decomposition):

1. Standardise / centre the data: $X_c = X - \text{mean}$ (column-wise).
2. Covariance matrix: $C = X_c^T X_c / (n - 1)$, size $d \times d$.
3. Eigenvalues and eigenvectors of C .
4. Sort by eigenvalue, largest first. Eigenvalue = variance along that direction.
5. Keep the top k eigenvectors W ($d \times k$). Project: $Z = X_c \cdot W$ ($n \times k$).

SVD: factor the centred data directly: $X_c = U \cdot S \cdot V^T$. The rows of V^T are the same principal directions, and $S^2 / (n - 1)$ are the same eigenvalues. Project: $Z = X_c \cdot V^T[:k]^T$ (equivalently $U[:, :k] \cdot S[:, :k]$).

PCA vs SVD in one sentence: PCA builds the covariance matrix first and eigen-decomposes it; SVD factorises the data matrix directly, which is numerically more stable and is how sklearn computes PCA internally. `TruncatedSVD` does NOT centre the data, so it also works on sparse matrices.

From scratch with NumPy (verified: matches sklearn exactly)

```
import numpy as np

def pca_scratch(X, k):
    Xc = X - X.mean(axis=0)                # 1. centre
    C = np.cov(Xc, rowvar=False)           # 2. covariance (d x d)
    vals, vecs = np.linalg.eigh(C)         # 3. eigh: for symmetric
    matrices
    idx = np.argsort(vals)[::-1]           # 4. sort descending
    vals, vecs = vals[idx], vecs[:, idx]
    ratio = vals / vals.sum()              # explained variance
    ratio
    return Xc @ vecs[:, :k], ratio         # 5. project

def svd_scratch(X, k):
    Xc = X - X.mean(axis=0)
    U, S, Vt = np.linalg.svd(Xc, full_matrices=False)
    ratio = S**2 / np.sum(S**2)
    return Xc @ Vt[:k].T, ratio
```

Dry run on a tiny example: two features where $x_2 \approx 2 \cdot x_1$. Almost all points lie on one line, so eigenvalue 1 is huge and eigenvalue 2 is tiny (ratio roughly [0.99, 0.01]). Keeping $k = 1$ loses about 1% of the variance and halves the columns.

Sign note: components may come out with flipped signs compared to sklearn. That is fine — a direction and its opposite are the same axis.

sklearn version + re-running the classifiers (Assignment 3 Q2)

```
from sklearn.decomposition import PCA, TruncatedSVD

# choose k from the cumulative variance plot (X_train already scaled)
pca_full = PCA().fit(X_train)
cum = np.cumsum(pca_full.explained_variance_ratio_)
plt.plot(range(1, len(cum) + 1), cum, marker='o'); plt.axhline(0.95,
color='r', ls='--')
plt.xlabel('Number of components'); plt.ylabel('Cumulative explained
variance'); plt.show()
k = np.argmax(cum >= 0.95) + 1
print('k =', k)

pca = PCA(n_components=k)
```

```

Xtr_pca = pca.fit_transform(X_train); Xte_pca = pca.transform(X_test)    # fit
on train only!

svd = TruncatedSVD(n_components=k, random_state=42)
Xtr_svd = svd.fit_transform(X_train); Xte_svd = svd.transform(X_test)

res_pca = evaluate(models, Xtr_pca, Xte_pca, y_train, y_test, '(PCA)')
res_svd = evaluate(models, Xtr_svd, Xte_svd, y_train, y_test, '(SVD)')

# comparison table
for df_, tag, n in [(results_orig, 'Original', X_train.shape[1]), (res_pca,
'PCA', k), (res_svd, 'SVD', k)]:
    df_['Feature set'] = tag; df_['#Features'] = n
comp = pd.concat([results_orig, res_pca, res_svd])
print(comp.pivot(index='Model', columns='Feature set', values='ROC-AUC'))
print(comp.pivot(index='Model', columns='Feature set', values='Train time
(s)'))
comp.pivot(index='Model', columns='Feature set',
values='F1').plot(kind='bar'); plt.ylabel('F1'); plt.show()

# optional: 2-D scatter to visualise class separation
Z2 = PCA(2).fit_transform(X_train)
plt.scatter(Z2[:, 0], Z2[:, 1], c=y_train, cmap='coolwarm', s=4);
plt.xlabel('PC1'); plt.ylabel('PC2'); plt.show()

```

What to expect and conclude: with 95% variance kept, scores usually stay within about 0.01–0.02 of the original while training gets faster (most visible for SVM and KNN). Tree models can lose a bit because they liked the original, interpretable columns. Conclusion: "Dimensionality reduction maintained (or slightly decreased) performance with $N \rightarrow k$ features and lower training time; useful when speed or multicollinearity matters, not when interpretability matters."

Viva traps: always scale before PCA (otherwise TotalCharges in thousands dominates); fit PCA on training data only; PCA is unsupervised — it never looks at y .

Module 6: Ridge, Lasso, Elastic Net (House Sales)

Regularization adds a penalty on big coefficients to the linear-regression loss. Ridge shrinks all coefficients, Lasso can push some to exactly zero (automatic feature selection), Elastic Net mixes both. In sklearn, the lambda in your notes is called `alpha`.

Intuition first

Analogy: A student who memorises every past paper (overfitting) does badly on a new paper. Regularization is a teacher saying "keep your answers simple" — every extra mark of complexity costs points. A little penalty helps you generalise; too much and you write nothing useful (underfitting).

Model	Loss minimised (plain)	Effect on coefficients
Linear	sum of $(y - \hat{y})^2$	No restriction; can blow up with correlated features
Ridge (L2)	sum of $(y - \hat{y})^2 + \lambda \cdot \text{sum of } b^2$	All shrink smoothly towards 0, none become exactly 0
Lasso (L1)	sum of $(y - \hat{y})^2 + \lambda \cdot \text{sum of } \text{abs}(b)$	Some become exactly 0 $\rightarrow$ feature selection
Elastic Net	errors + $\lambda \cdot [r \cdot \text{sum } \text{abs}(b) + (1 - r)/2 \cdot \text{sum } b^2]$	Mix; <code>l1_ratio = r</code> . Handles groups of correlated features better than Lasso

Behaviour as λ grows: $\lambda = 0 \rightarrow$ same as Linear. Small $\lambda \rightarrow$ train error rises a little, test error often falls (less overfitting). Huge $\lambda \rightarrow$ all coefficients near 0 $\rightarrow$ both errors high (underfitting). The best λ is at the bottom of the TEST error curve.

Why Lasso gives exact zeros: the L1 penalty has a sharp corner at 0 (abs function), so the optimum often lands right on it. The L2 penalty (b^2) is smooth there, so it only gets close.

Scaling is mandatory: the penalty treats every coefficient equally, so features must be on the same scale or the penalty unfairly hits small-unit features.

Dataset quirks (Kaggle "House Sales in King County" — `kc_house_data.csv`)

- Drop `id`. Convert `date` (e.g. '20141013T000000') $\rightarrow$ extract year/month or drop it.
- `zipcode` is a category, not a number $\rightarrow$ one-hot it or drop it.
- `price` is heavily right-skewed $\rightarrow$ optionally model `np.log1p(price)` and mention why.
- `yr_renovated = 0` means never renovated $\rightarrow$ feature `renovated = (yr_renovated > 0).astype(int)`; `house_age = 2015 - yr_built`.
- If your copy has a different house dataset (e.g. the Kaggle "House Prices – Advanced Regression"), the same template applies: fill numeric NaNs with median, categorical with mode, then `get_dummies`.

Template

```
import pandas as pd, numpy as np, matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

df = pd.read_csv('kc_house_data.csv')
df['date'] = pd.to_datetime(df['date'])
df['sale_year'] = df['date'].dt.year
df['house_age'] = df['sale_year'] - df['yr_built']
df['renovated'] = (df['yr_renovated'] > 0).astype(int)
df = df.drop(['id', 'date', 'zipcode', 'yr_renovated'], axis=1)
df = df.fillna(df.median(numeric_only=True))
df = pd.get_dummies(df, drop_first=True, dtype=int)      # if any text columns
remain

X = df.drop('price', axis=1); y = df['price']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
sc = StandardScaler(); X_train = sc.fit_transform(X_train); X_test =
sc.transform(X_test)

def metrics(y_true, y_pred):
    mse = mean_squared_error(y_true, y_pred)
    return mse, np.sqrt(mse), mean_absolute_error(y_true, y_pred),
r2_score(y_true, y_pred)

# 1. Baseline linear regression
lr = LinearRegression().fit(X_train, y_train)
print('Linear:', metrics(y_test, lr.predict(X_test)))

# 2. Lambda sweep for the three regularized models
alphas = [0.001, 0.01, 0.1, 1, 10, 100, 1000]
makers = {'Ridge':      lambda a: Ridge(alpha=a),
          'Lasso':      lambda a: Lasso(alpha=a, max_iter=10000),
          'ElasticNet': lambda a: ElasticNet(alpha=a, l1_ratio=0.5,
max_iter=10000)}

rows = []
for name, make in makers.items():
    for a in alphas:
```



```

        m = make(a).fit(X_train, y_train)
        tr_mse = mean_squared_error(y_train, m.predict(X_train))
        te = metrics(y_test, m.predict(X_test))
        rows.append([name, a, tr_mse, *te, np.sum(m.coef_ == 0)])
sweep = pd.DataFrame(rows, columns=['Model', 'alpha', 'Train MSE', 'Test
MSE', 'RMSE', 'MAE', 'R2', '#zero coefs'])
print(sweep)

# 3. Train vs test error against lambda (log x-axis)
fig, ax = plt.subplots(1, 3, figsize=(16, 4))
for i, name in enumerate(makers):
    s = sweep[sweep.Model == name]
    ax[i].plot(s.alpha, s['Train MSE'], 'o-', label='Train')
    ax[i].plot(s.alpha, s['Test MSE'], 's-', label='Test')
    ax[i].set_xscale('log'); ax[i].set(title=name, xlabel='lambda (alpha)',
ylabel='MSE'); ax[i].legend()
plt.show()

# 4. Best alpha per model -> final comparison table + coefficient plot
best = sweep.loc[sweep.groupby('Model')['Test MSE'].idxmin()]
print(best)
coefs = pd.DataFrame({'Linear': lr.coef_}, index=X.columns)
for _, r in best.iterrows():
    coefs[r.Model] = makers[r.Model](r.alpha).fit(X_train, y_train).coef_
coefs.plot(kind='bar', figsize=(14, 5)); plt.ylabel('Coefficient');
plt.title('Coefficients by model'); plt.show()

```

Note on the alpha range: with price in dollars, Lasso may need large alphas (100–10,000) before coefficients hit zero. If nothing changes across your sweep, extend it: `alphas = np.logspace(-3, 4, 8)`.

What to write: "As λ increases, Ridge coefficients shrink smoothly; Lasso sets k coefficients to zero at $\lambda = X$ (it dropped features A, B); Elastic Net sits between. Test error is lowest at $\lambda = X$, and the regularized models perform about the same as plain linear regression because n is large relative to the number of features — regularization mainly improved stability and simplicity rather than accuracy."

Module 7: Hypothesis testing — independent vs paired t-test

Both tests ask "is the difference between two means real or just noise?". Use **independent** when the two groups are different people, **paired** when the same people are measured twice. With your assignment data: Q1 is significant ($p \approx 0.00004$), Q2 is significant ($p \approx 0.00000003$).

Intuition first

t-statistic = signal / noise = (difference in means) / (standard error of that difference). Big $t \rightarrow$ the difference is large compared with the natural scatter.

p-value: if there were truly no difference (H_0), the probability of seeing a t this extreme by chance. Small $p \rightarrow H_0$ is hard to believe.

Decision rule (write this every time): H_0 : mean1 = mean2. H_1 : mean1 $\neq$ mean2 (two-tailed). If $p < \alpha$ (0.05) $\rightarrow$ reject $H_0 \rightarrow$ significant difference. Otherwise $\rightarrow$ fail to reject H_0 (never say "accept H_0 ").

Why pairing matters (analogy): Comparing a student's marks before and after coaching. Students differ hugely from each other (a 50-scorer vs a 70-scorer), but each student's own improvement is consistent. The paired test looks only at each student's change, removing the between-student scatter from the noise. Less noise $\rightarrow$ bigger t .

Q1 — Independent two-sample t-test

```
from scipy import stats
group_A = [65, 70, 68, 72, 66, 75, 69, 71, 67, 70]
group_B = [72, 75, 78, 74, 80, 77, 73, 79, 76, 81]
alpha = 0.05

t, p = stats.ttest_ind(group_A, group_B)          # equal_var=True by default
(Student's t)
print(f't = {t:.3f}, p = {p:.6f}')
print('Reject H0: significant difference' if p < alpha else 'Fail to reject
H0: no significant difference')
```

Output: mean A = 69.3, mean B = 76.5, $t = -5.357$, $p = 0.0000432$, $df = 18$. Since $p < 0.05$, reject H_0 : teaching method B gives significantly higher scores.

Sign of t: negative because A's mean is smaller and A was passed first. Only the size matters for a two-tailed test.

Optional extras that earn marks: check normality with `stats.shapiro(group_A)` ($p > 0.05 \rightarrow$ roughly normal) and equal variances with `stats.levene(group_A, group_B)` ($p > 0.05 \rightarrow$ equal). If Levene fails, use `equal_var=False` (Welch's t-test); here it gives the same conclusion ($p \approx 0.0000432$).

Q2 — Paired t-test

```
before = [52, 48, 65, 60, 55, 70, 58, 62, 50, 67]
after  = [60, 55, 70, 66, 63, 75, 64, 68, 57, 72]

t, p = stats.ttest_rel(after, before)
print(f't = {t:.3f}, p = {p:.10f}')
```

Full dry run (do this once by hand):

- Differences (after – before): 8, 7, 5, 6, 8, 5, 6, 6, 7, 5.
- Mean difference $\bar{d} = 63 / 10 = 6.3$.
- Squared deviations from 6.3: 2.89, 0.49, 1.69, 0.09, 2.89, 1.69, 0.09, 0.09, 0.49, 1.69 $\rightarrow$ sum 12.1.
- $s_d = \sqrt{12.1 / 9} = 1.159$. Standard error = $1.159 / \sqrt{10} = 0.367$.
- $t = 6.3 / 0.367 = 17.18$, $df = 9 \rightarrow p = 3.45 \times 10^{-8}$.

Conclusion: $p < 0.05 \rightarrow$ reject H_0 ; the training program significantly increased programming scores (by 6.3 points on average).

Q3 — Why the two tests give different answers

Run BOTH tests on the same before/after data — this is the cleanest demonstration:

```
t_ind, p_ind = stats.ttest_ind(after, before)    # wrongly treats them as
different people
t_rel, p_rel = stats.ttest_rel(after, before)     # correctly pairs each
student
print(f'Independent: t = {t_ind:.3f}, p = {p_ind:.4f}')
```

```
print(f'Paired:      t = {t_rel:.3f}, p = {p_rel:.2e}')
```

Test	t	p	Decision at $\alpha = 0.05$
Independent	2.019	0.0586	Not significant
Paired	17.182	0.0000000345	Significant

Explanation to write: The mean difference (6.3) is the same in both. The independent test's noise term uses the spread of scores between students (SDs about 7.4 and 6.5), so the improvement looks small relative to it. The paired test uses the spread of each student's own change (SD 1.16), which is tiny because everyone improved by 5–8 points. Smaller standard error $\rightarrow$ much larger t $\rightarrow$ much smaller p. Using the wrong test here leads to a wrong conclusion. Rule: same subjects measured twice $\rightarrow$ paired; different subjects $\rightarrow$ independent.

Quick reference — which test?

- Two separate groups $\rightarrow$ `stats.ttest_ind(a, b)`
- Same subjects, two measurements $\rightarrow$ `stats.ttest_rel(after, before)`
- One group vs a known value (e.g. mean = 70) $\rightarrow$ `stats.ttest_1samp(a, 70)`
- One-tailed (e.g. "after > before") $\rightarrow$ add `alternative='greater'`

Cheat sheet: imports, errors, viva questions

Read this section last tonight and first thing tomorrow morning.

Master import cell (paste at the top of every notebook)

```
import requests, json, time, re
from bs4 import BeautifulSoup
import numpy as np, pandas as pd
import matplotlib.pyplot as plt, seaborn as sns
from scipy import stats
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.linear_model import LinearRegression, LogisticRegression, Ridge,
Lasso, ElasticNet
from sklearn.feature_selection import RFE
from sklearn.decomposition import PCA, TruncatedSVD
from sklearn.metrics import *
import warnings; warnings.filterwarnings('ignore')
```

Common errors and fixes

Error	Cause	Fix
<code>AttributeError: 'NoneType' object has no attribute 'text'</code>	<code>find()</code> found nothing	Use the safe <code>get_text()</code> helper; check tag/class spelling in the page source
403 Forbidden (IMDb)	Site blocks scripts	Add <code>headers={'User-Agent': 'Mozilla/5.0'}</code> ; else use a saved HTML / Kaggle CSV
<code>ValueError: could not convert string to float: '£51.77'</code>	Currency symbol left in	<code>.str.replace('£', '').str.replace('Â', '')</code>
<code>ValueError: Expected 2D array, got 1D array</code>	Passed <code>df['col']</code> to a scaler	Use <code>df[['col']]</code> (double brackets)
<code>ValueError: could not convert string to float: 'Male'</code>	Text columns into a model	<code>pd.get_dummies(df, drop_first=True)</code> first
<code>ValueError: Input contains NaN</code>	Missing values left	<code>df.isnull().sum()</code> then fill
<code>ConvergenceWarning (Logistic / Lasso)</code>	Not enough iterations or unscaled data	Scale; <code>max_iter=1000</code> or <code>10000</code>
<code>AttributeError: predict_proba on SVC</code>	Probabilities off by default	<code>SVC(probability=True)</code>
<code>UnicodeDecodeError</code> reading a file	Wrong encoding	<code>open(f, encoding='utf-8')</code>
<code>KeyError: 'col'</code>	Column name has a space or different case	<code>print(df.columns.tolist()); df.columns = df.columns.str.strip()</code>

Decision cheat sheet

- **Fill numeric NaN:** median (skewed / outliers) or mean (symmetric).
- **Fill categorical NaN:** mode or 'Unknown'.
- **Scaler:** bounded, no outliers → Min-Max; general / linear models / PCA → Standard; heavy outliers → Robust or log first.
- **Outliers:** IQR rule ($1.5 \times \text{IQR}$ beyond $Q1/Q3$) → clip (cap) to keep rows, or remove if few and clearly errors.
- **Encoding:** nominal (no order: gender, contract) → one-hot; ordinal (Low < Medium < High, star rating) → map to numbers.
- **Regression metric to rank by:** R^2 (higher) or RMSE (lower). **Classification on imbalanced data:** ROC-AUC, F1, recall — not accuracy.
- **t-test:** different people → independent; same people twice → paired.

Likely viva questions (with one-line answers)

1. **Difference between structured, semi-structured and unstructured data?** Fixed schema table / tagged key-value like JSON / free text, images, raw HTML.
2. **Why convert to JSON before the DataFrame?** JSON keeps the scraped records as-is with flexible fields; it is a checkpoint and shows the semi-structured stage.
3. **Normalization vs standardization?** Min-Max rescales to $[0,1]$; Z-score centres to mean 0, std 1.
4. **Why split before scaling / fitting PCA?** To prevent data leakage from the test set.
5. **What does $R^2 = 0.6$ mean?** The model explains 60% of the variance in house values.
6. **MAE vs RMSE?** RMSE punishes large errors more because errors are squared.
7. **What is multicollinearity?** Features strongly correlated with each other (AveRooms and AveBedrms); makes coefficients unstable. Ridge helps.
8. **How does RFE work?** Repeatedly fits the model and removes the feature with the smallest importance until k remain.
9. **Precision vs recall?** Precision: of predicted churners, how many are real. Recall: of real churners, how many were caught.
10. **Why is accuracy misleading for churn?** About 74% of customers stay, so predicting "No" always scores ~74% while catching nobody.
11. **What does the ROC curve show?** TPR vs FPR across all thresholds; AUC summarises ranking quality.

12. **What is a principal component?** A new axis, a weighted combination of original features, along which the data varies most; components are uncorrelated.
13. **How do you choose the number of components?** Cumulative explained variance $\geq 90-95\%$ (elbow of the scree plot).
14. **Ridge vs Lasso?** L2 shrinks all; L1 can zero out coefficients $\rightarrow$ feature selection.
15. **What happens when lambda is too large?** All coefficients $\rightarrow 0$, underfitting, both train and test error rise.
16. **What is a p-value?** Probability of results at least this extreme if H_0 were true.
17. **Type I vs Type II error?** Rejecting a true H_0 (false alarm, probability α) vs failing to reject a false H_0 (miss).
18. **Assumptions of the t-test?** Roughly normal data, independent observations, (for Student's independent test) equal variances.
19. **Why does the paired test give a smaller p here?** It removes between-student variability, so the standard error is much smaller.
20. **Is web scraping always allowed?** Check `robots.txt` and terms of use; add delays; `books.toscrape` is a practice site built for it.

Final 15-minute checklist before you start coding tomorrow

- ☐ Read the question and list every deliverable (number of attributes, features, plots, insights, metrics).
- ☐ Paste the master import cell.
- ☐ Load data $\rightarrow$ `shape`, `head`, `info`, `isnull().sum()`, `duplicated().sum()`.
- ☐ Write one markdown line before every step.
- ☐ Finish with a comparison table, plots with titles, and numbered insights backed by numbers.